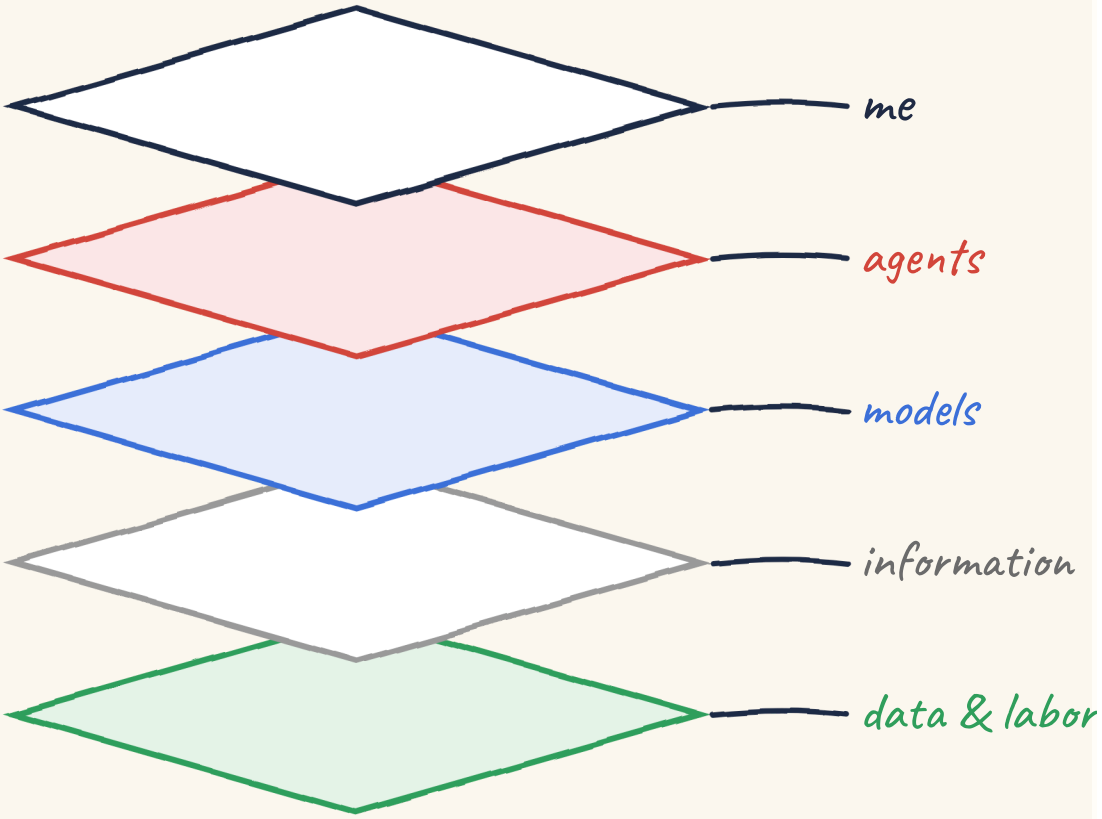


← binder clip (provided)

EXPERIMENTAL MIDTERM F26

CIXD 3110 · a zine about the tools that watch, learn & act

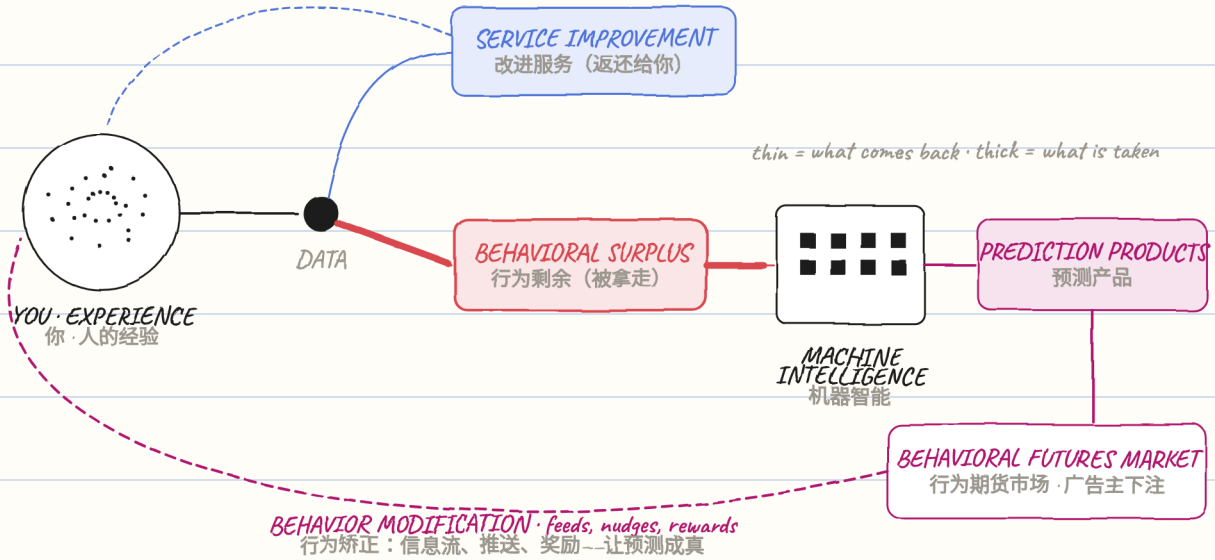


Name: _____

Section: 3110 / 6110

1.1 Surveillance capitalism

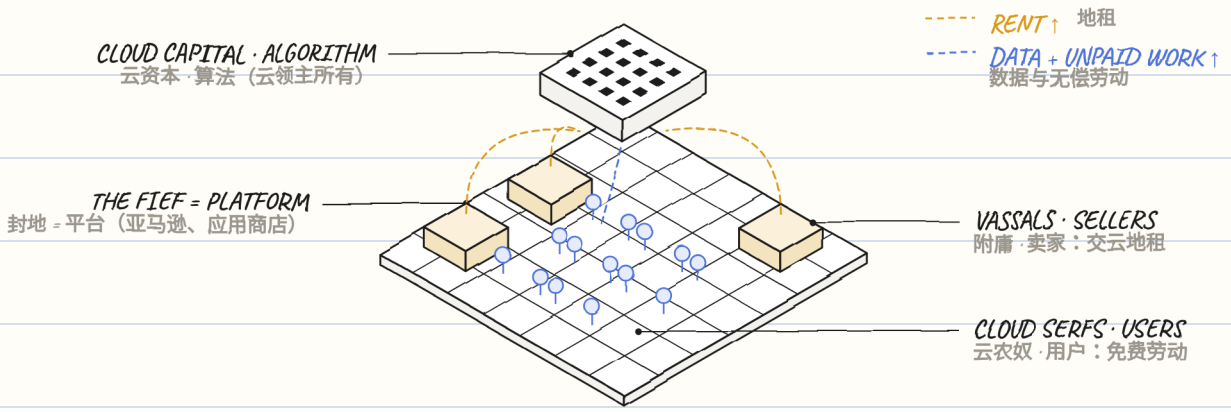
Describe surveillance capitalism.



Surveillance capitalism is Shoshana Zuboff's name for an economic order that claims human experience as free raw material. Platforms collect more than they need (Behavioral surplus), machine intelligence turns it into prediction products, and these are sold in behavioral futures markets to advertisers — then the system nudges behavior so the predictions come true (Behavior modification). It creates a huge asymmetry: they know us, we don't know them.

1.2 Technofeudalism

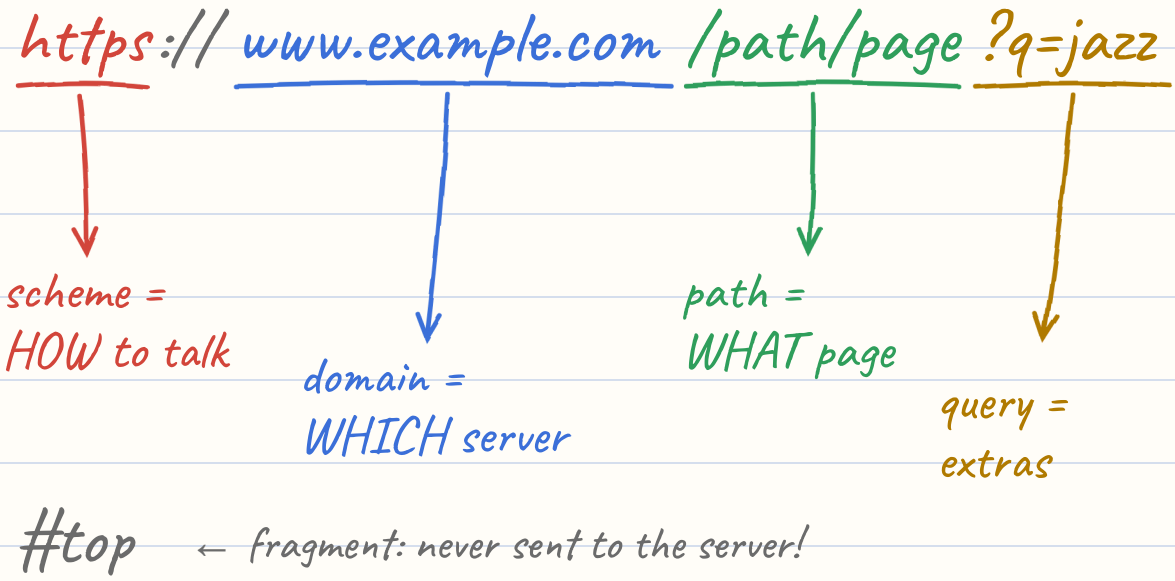
What does technofeudalism highlight? How does it relate to surveillance capitalism?



Technofeudalism (Varoufakis) highlights that platforms are no longer markets but fiefs: owners of Cloud capital earn rent, not profit — sellers pay Cloud rent to exist inside Amazon or an app store, and users are Cloud serfs whose unpaid posting and scrolling keeps building that capital. Relation: both start from the same extraction of behavior as data — surveillance capitalism describes the mechanism (extract → predict → modify); technofeudalism names the economic structure it builds.

1.3 URL

Name the parts of a URL and say what each one tells the server.

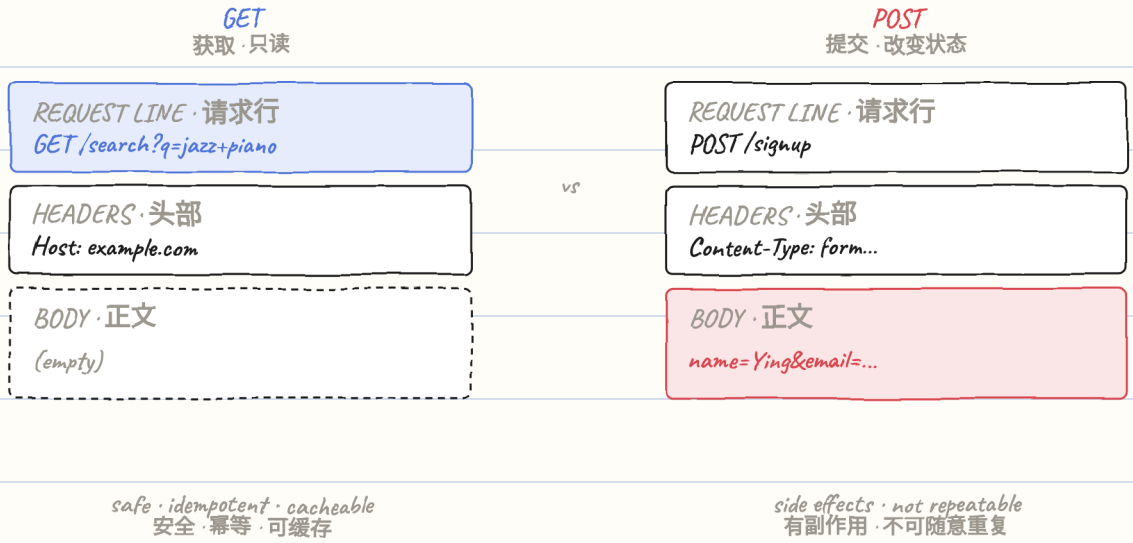


#top ← fragment: never sent to the server!

MDN: a URL is the address of a unique resource on the Web. Scheme (https) — which protocol to use; domain (www.example.com) — which server; port (:443) — which gate on it, usually implicit; path (/path/file.html) — which resource; query parameters (?key=value&...) — extra information the server uses; fragment (#section) — a spot in the page that is never sent to the server.

1.4 GET vs. POST

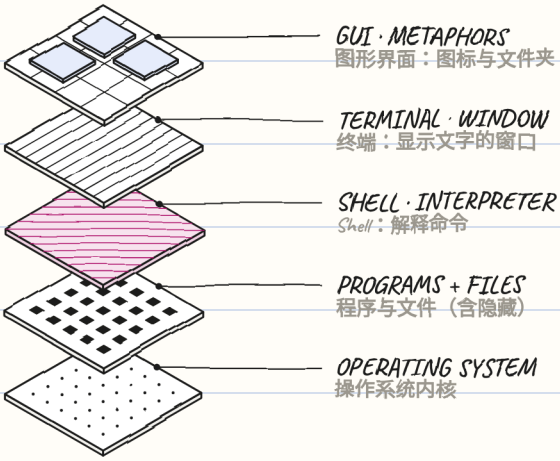
What is the difference between GET and POST, and when you fill in a form on a website, which one happens?

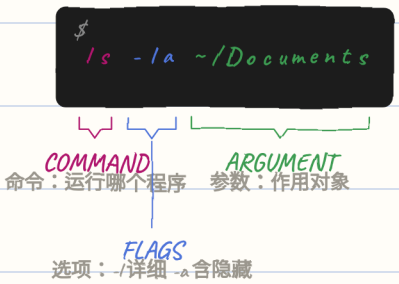


Both are HTTP request methods. GET asks the server to send back a resource and should only retrieve data — any data rides in the URL (visible, cached, bookmarkable, repeatable). POST submits data to be processed, often changing the server's state — the data travels in the request body. When you fill in a form (log in, sign up, check out), it is usually POST; simple search boxes use GET, which is why "?q=" shows in the address bar.

1.5 Command line / shell

What is a shell, and what does the terminal show you that the GUI hides?



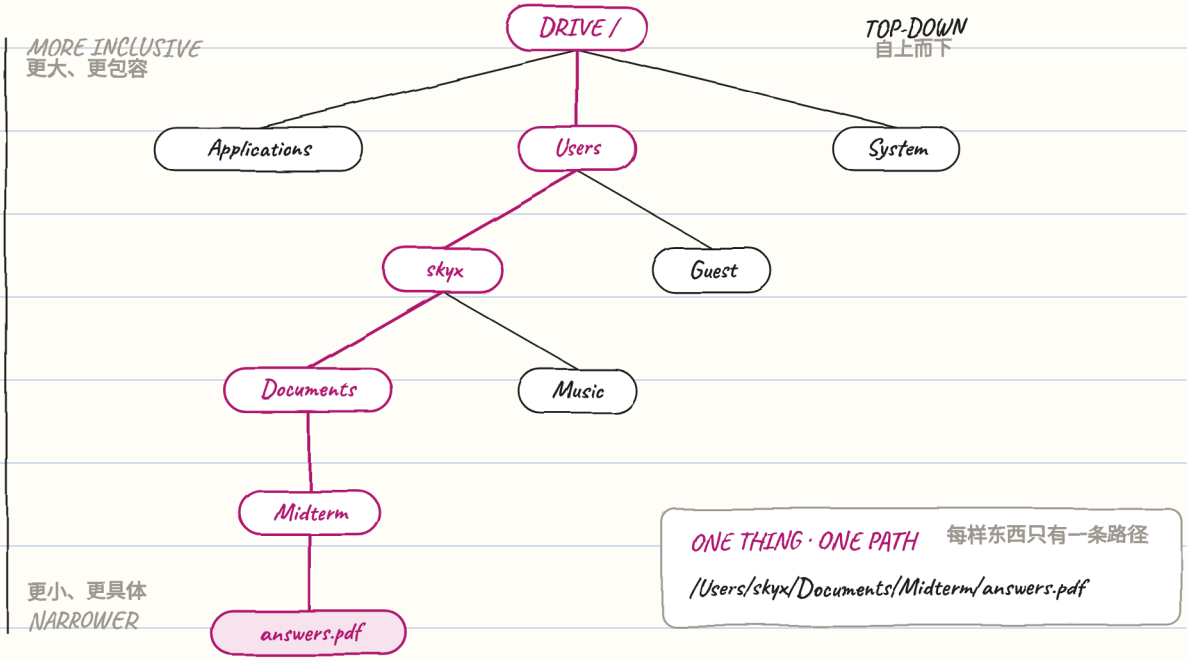


EXPLAINSHELL-STYLE BREAKDOWN

A Shell is a program that reads the text commands you type, interprets them and asks the operating system to run programs (bash, zsh); the Terminal is just the window that shows it. The terminal shows what the GUI hides: the actual command behind an action (command · flags · arguments), the real file structure (full paths, hidden dot-files, permissions, processes), raw output and errors, and the ability to combine tools. Stephenson's point: the GUI is a layer of metaphors that makes things easy by hiding the machinery.

2.1 Hierarchy

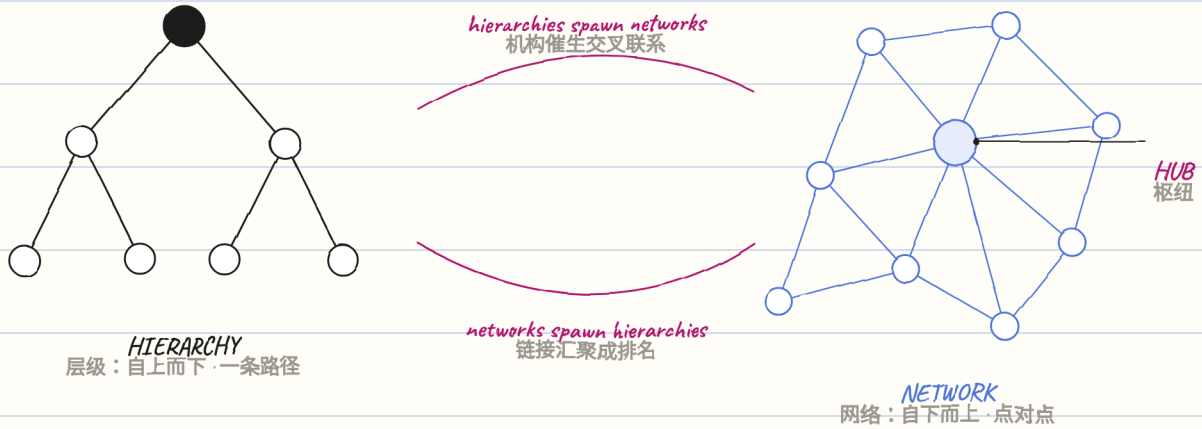
Wright's definition of a hierarchy, and one example from software you use every day.



Wright: a Hierarchy is a top-down structure that orders things by inclusiveness — broad categories contain narrower ones, like a tree, so each item has one place. Everyday example: my computer's file system — Drive → Users → Documents → Midterm → answers.pdf; every folder has one parent and every file one path.

2.2 Network

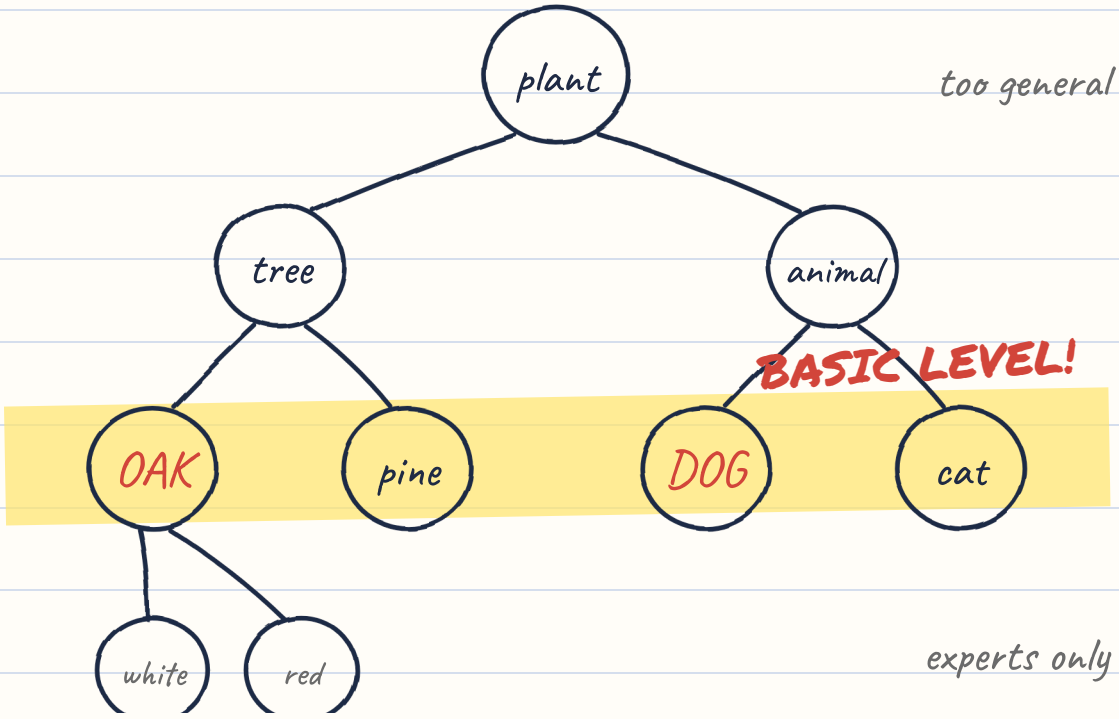
Wright's definition of a network, and what he means when he says networks and hierarchies give rise to each other.



Wright: a Network is a bottom-up, distributed web of peer-to-peer links with no single top; a hierarchy is a top-down tree. They give rise to each other: networks grow hubs and rankings that harden into new hierarchies (links on the Web → Google's ranking), and every hierarchy sprouts side-connections that form networks ("see also" cross-references, informal ties in an org chart).

2.3 Folk taxonomy

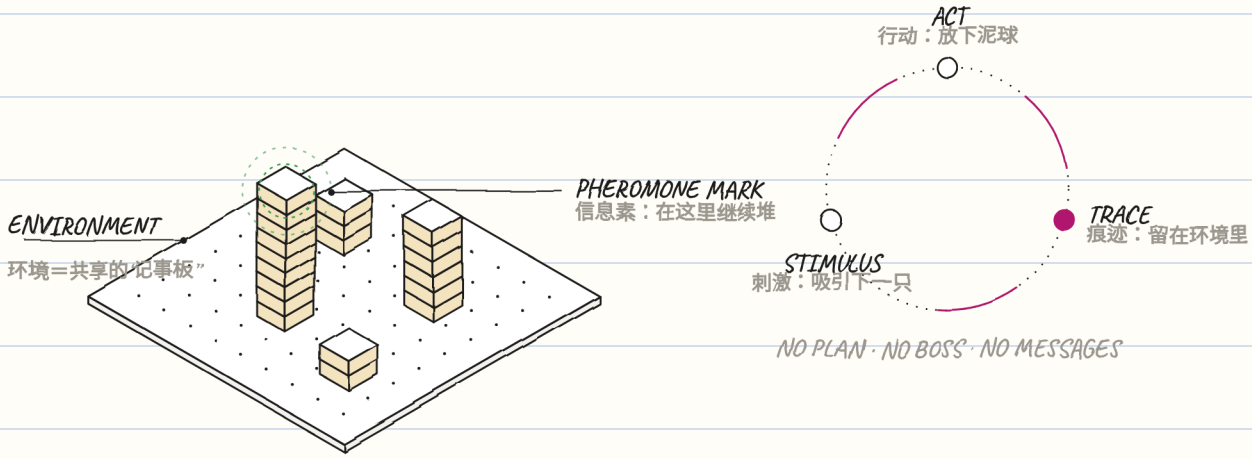
What is a folk taxonomy, and why does the psychologically primary category (oak, dog) sit in the middle of it rather than at the top?



A Folk taxonomy is a culture's everyday, unwritten way of naming and grouping living things; Berlin found the same few ranks everywhere (plant → tree → oak → white oak). The psychologically primary category sits in the middle because it is Rosch's Basic level (generic rank): the level built from perception and use — learned first, named fastest, shortest words. Levels above are abstractions; levels below are specialist detail.

2.4 Stigmergy

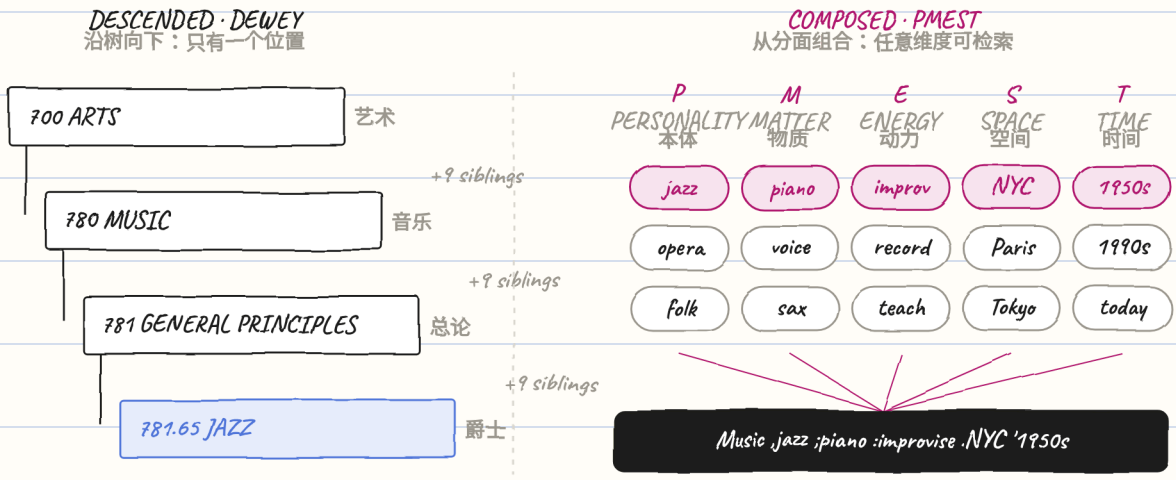
Define stigmergy from the termites, then give a human or digital example.



Stigmergy (Grassé, 1959, studying termites) is indirect coordination through traces left in the environment: a termite drops a pheromone-laced pellet, the mark stimulates the next termite to build in the same spot, and pillars and arches emerge with no plan, leader or direct communication. Digital example: Wikipedia — each edit (or "citation needed" tag) is a trace that prompts the next editor.

2.5 Faceted classification

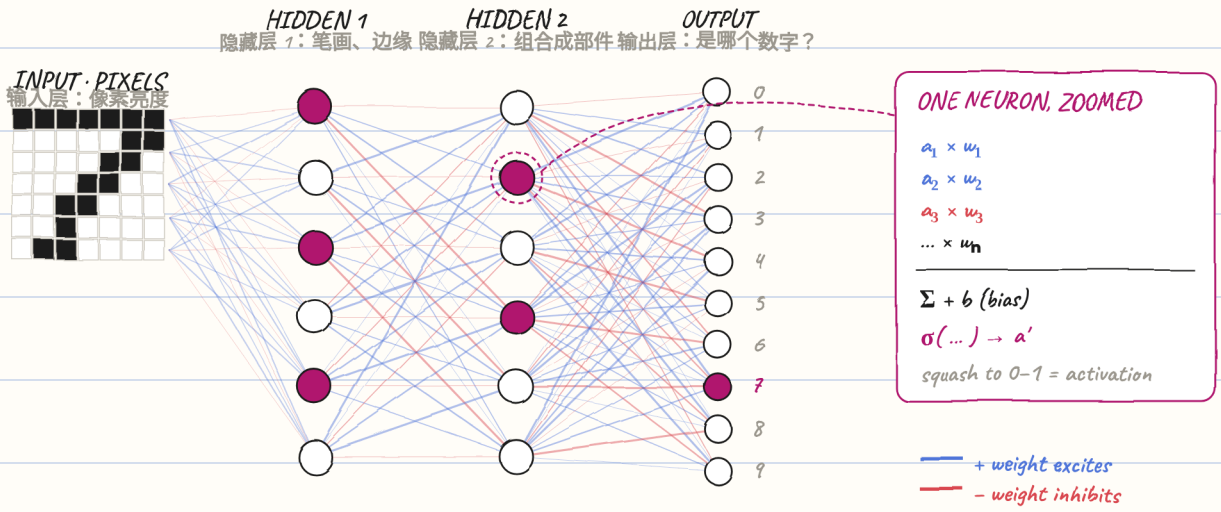
What changes when a classification is composed (UDC, Ranganathan's PMEST) rather than descended (Dewey)? Why does this matter for tagging and search?



Descended (Dewey): one pre-listed top-down tree — you climb down 700 → 780 → 781.65 and each item gets *one place*. *Composed* (UDC, Ranganathan's Colon Classification): the description is *built from independent facets* — PMEST = Personality, Matter, Energy, Space, Time — joined by signs, so new subjects and relations can be expressed. For tagging and search this means *one item, many access points*: tags are facets, and filter sidebars let you combine them in any order.

3.1 Neural network

Layers, weights, activations: in your own words, what does one layer do to the layer before it?

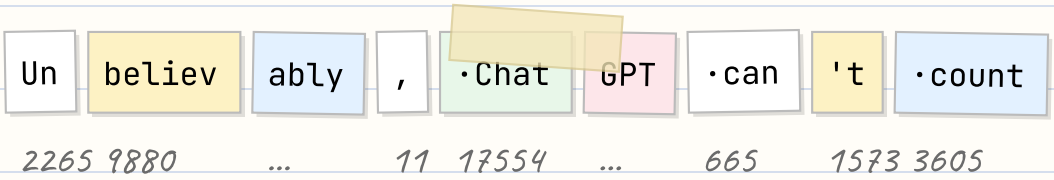


Each neuron holds an Activation (a number 0–1); a layer is a column of them. One layer takes all the activations of the layer before it, multiplies each by a Weight, adds them up with a Bias, and squashes the sum (sigmoid/ReLU) into its own activations: $a' = \sigma(W \cdot a + b)$. So each layer re-describes the previous one at a more abstract level (pixels → edges → strokes → "7"), and learning means adjusting the weights.

3.2 Token / tokenizer

What is a token, and why does the model see your sentence as tokens rather than words?

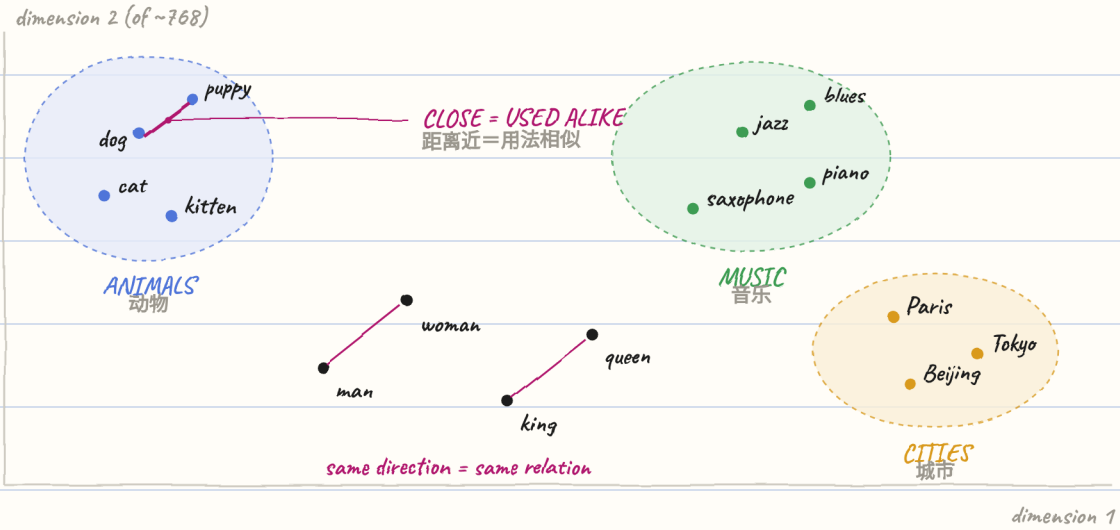
I cut my sentence with scissors:



A Token is a chunk of text — a word, part of a word, punctuation or a space — that the Tokenizer maps to an integer ID from a fixed Vocabulary. The model sees your sentence as tokens because a neural network only takes numbers from a fixed, finite list; sub-word pieces can spell any word (new words, names, typos, other languages) without an endless word list. That's also why limits and prices are counted in tokens.

3.3 Embedding

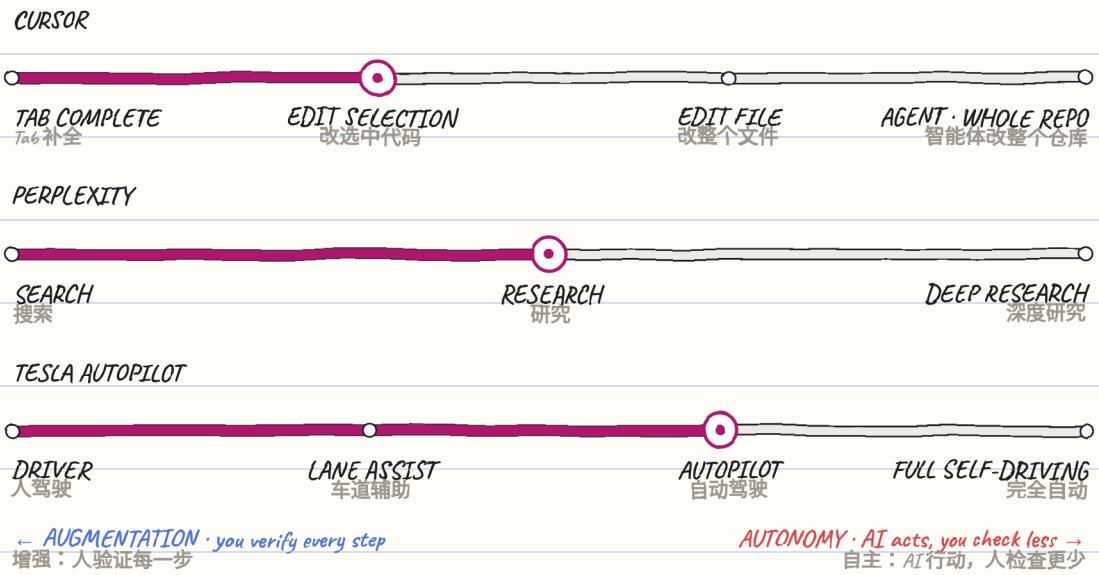
What is an embedding, and what does it mean that two words are close in that space?



An Embedding is a vector — a long list of numbers — that places a token or word as a point in a high-dimensional space, learned during training from the contexts the word appears in. Two words being close means the model has seen them used in similar contexts (similar usage, usually similar meaning — "you shall know a word by the company it keeps"); directions can also encode relations (man → woman ≈ king → queen).

3.4 Autonomy slider

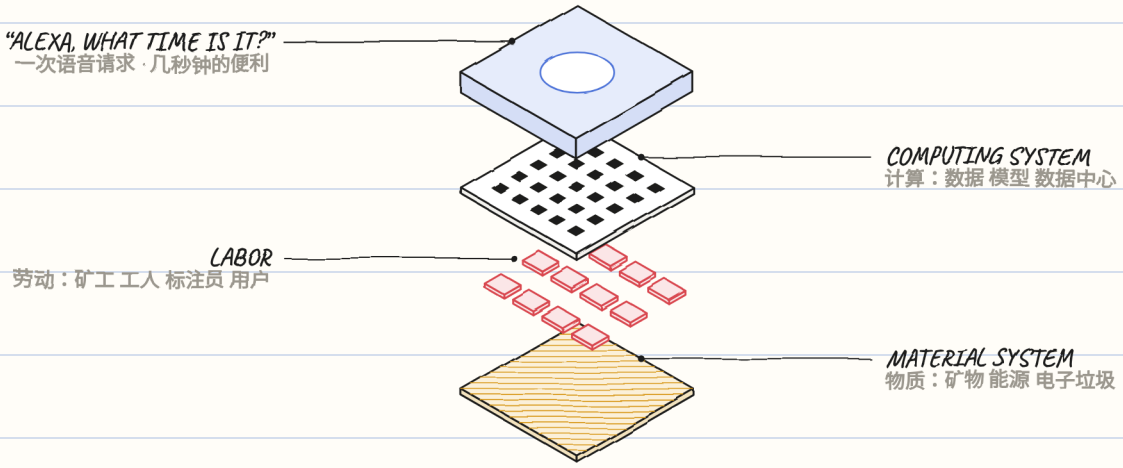
Karpathy's idea: what moves along the slider, and where would you set it for your own work right now?



Karpathy (Software 3.0) says good AI apps offer partial autonomy with an Autonomy slider: what moves is the split of control between human and AI — how big a chunk the AI does per step and how much the human must verify (Cursor: tab-complete → edit selection → edit file → agent on the whole repo; Perplexity: search → research → deep research). His advice: keep the AI "on a leash" and the Generation–verification loop fast. My setting: [name a level + a task + why — e.g. low/middle for design work, because taste and accuracy are hard to verify in big chunks].

3.5 Infrastructure as physical

From Crawford & Joler: what does it mean that an AI system is a material system, a form of labor, and a computing system at once?

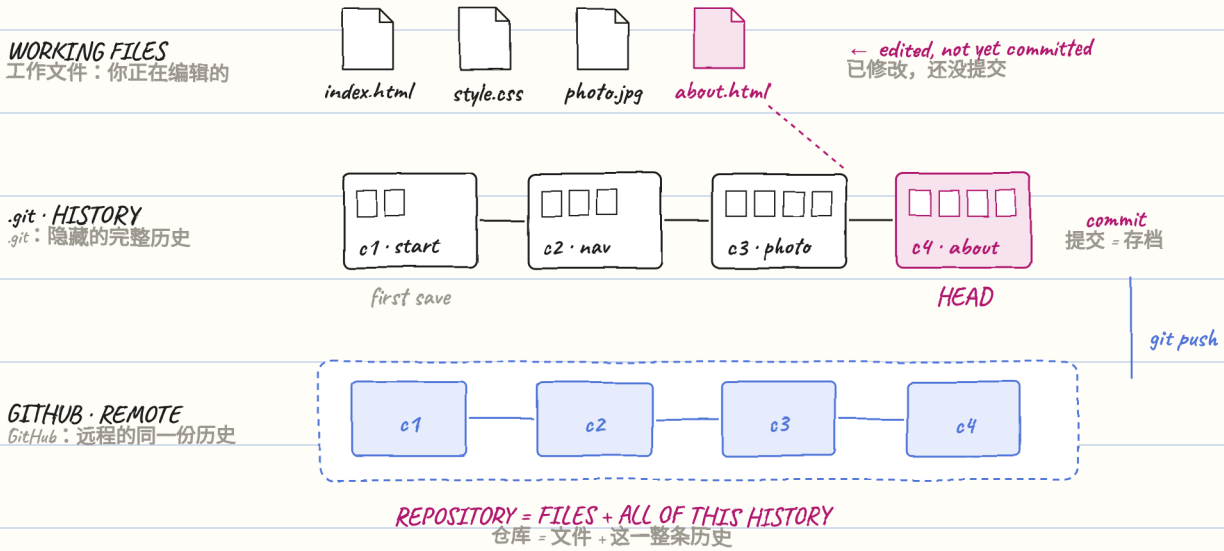


ONE ECHO REQUEST, TAKEN APART

In Anatomy of an AI System, Crawford & Joler trace one Amazon Echo request and show it is three systems working at the same time: a material system (minerals mined, energy, shipping, e-waste — Extractivism), a form of labor (miners, factory workers, data labelers, and users whose speech trains it — Digital labor), and a computing system (data, models, cloud servers). "At once" means every second of convenience consumes the earth and hidden human work as well as computation — the clean interface hides the first two.

4.1 Repository

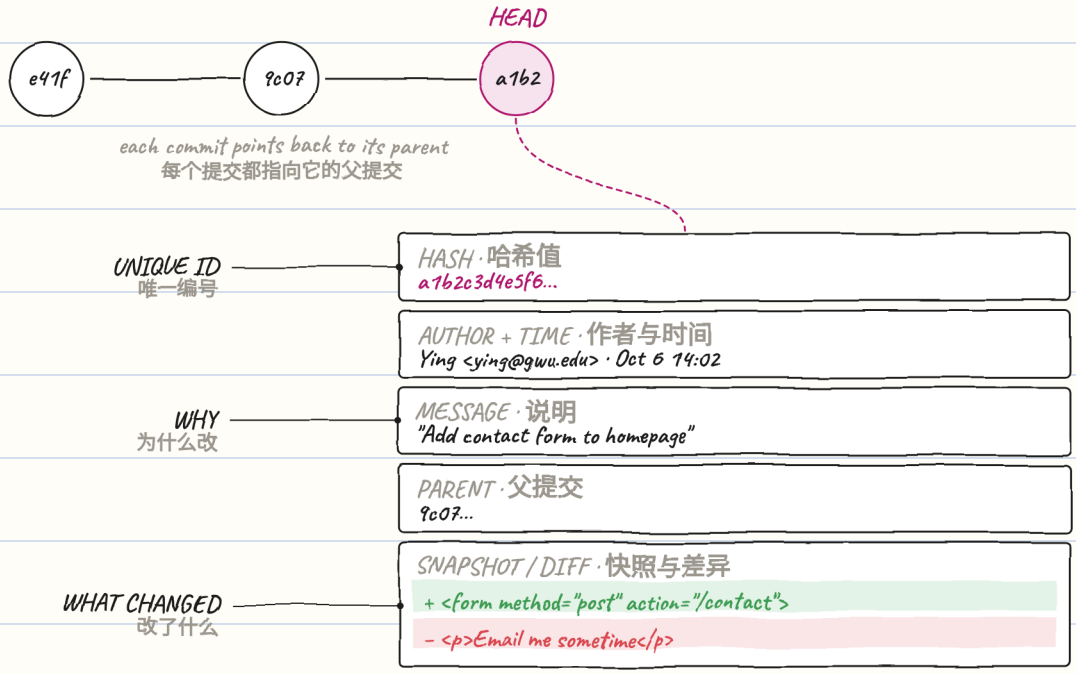
What is a repository?



A Repository (repo) is a project folder tracked by version control — the files plus their complete history. Git stores that history in a hidden `.git` folder as a chain of commits (snapshots with who, when and why), and a copy usually lives remotely on GitHub so others — and services like Vercel — can read it. The history is what makes it a repository rather than just a folder.

4.2 Commit

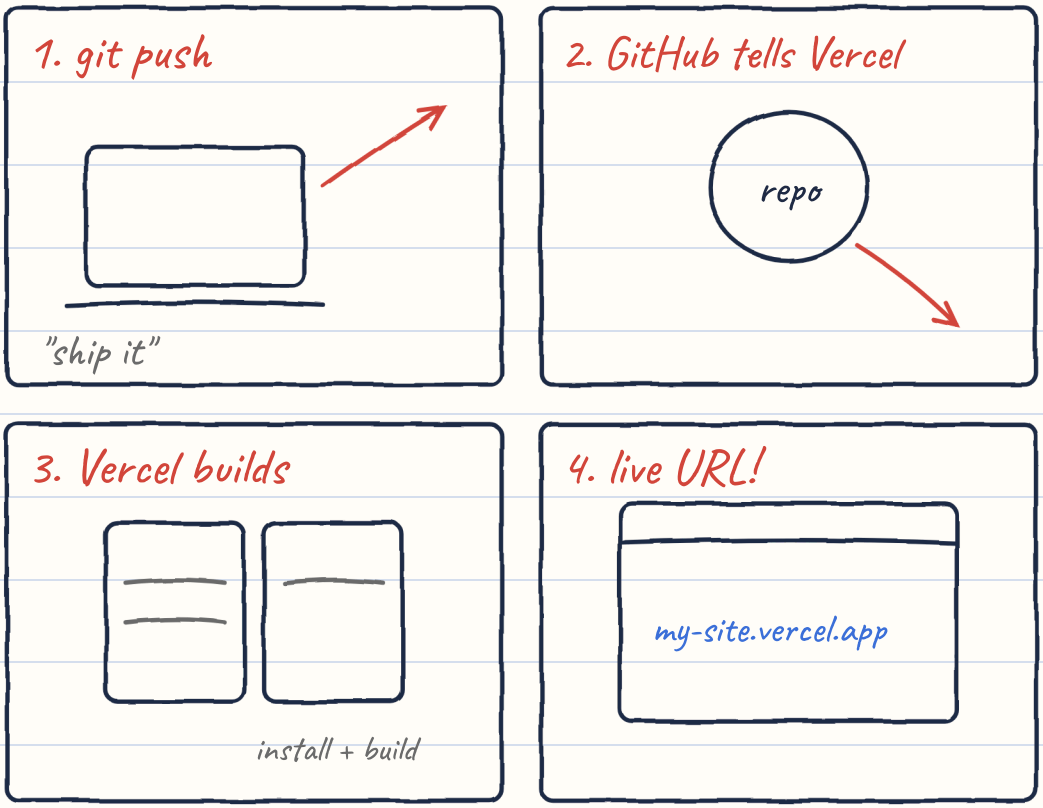
What is a commit?



A Commit is a saved snapshot of the repository at one moment, made on purpose, the basic unit of Git history. It records the changes (shown as a Diff), a unique Commit hash, the author and time, a message saying what and why, and a pointer to its parent commit — which links snapshots into a timeline you can compare against or return to. Committing saves locally; you Push to send commits to GitHub.

4.3 Deployment

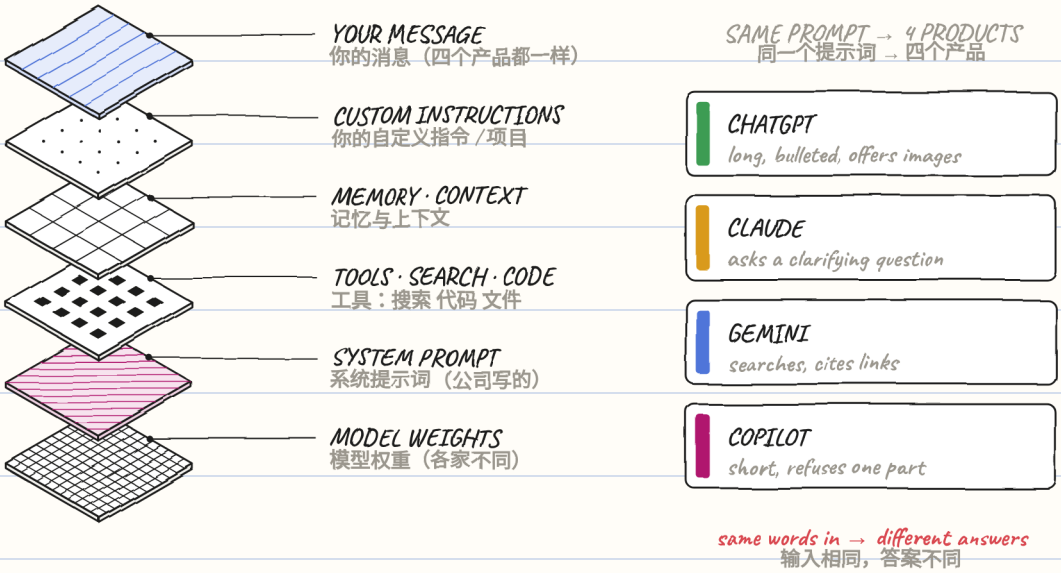
What happens between a push to GitHub and a page appearing on Vercel?



Because the repo is connected to Vercel, a push starts an automatic deployment: GitHub updates the branch → notifies Vercel (Webhook / Git integration) → Vercel clones that commit → runs the Build (install + build command) → uploads the output to its servers/CDN / edge network → assigns a URL (a preview URL per push; the production domain for the main branch). If the build fails, nothing is published and the old version stays live.

4.4 System prompt

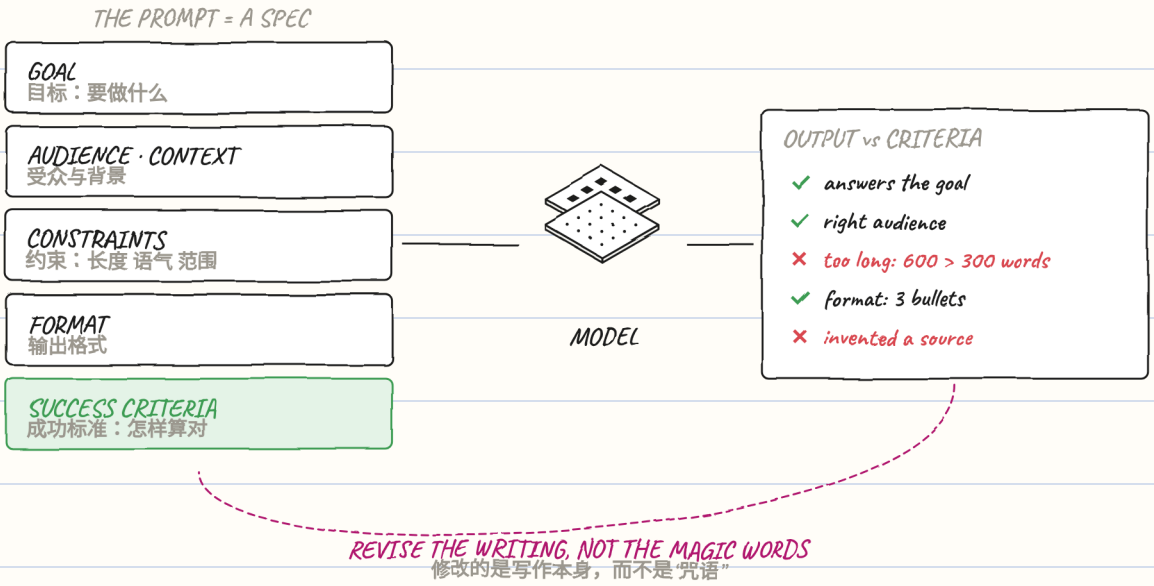
What is a system prompt, who wrote it, and what did you notice when the same prompt went to four products?



A System prompt is the hidden set of instructions placed before the conversation that sets the model's role, tone, rules, format and tool use (the API's `system` field). It is written by the company that builds the product (OpenAI, Anthropic, Google, Microsoft), not by you — you can only add your own layer with Custom instructions / Projects. In the Four Prompts activity the same prompt gave clearly different answers (length, tone, caution, searching, citing) because each product wraps the model in a different system prompt, tools, memory and model: Product ≠ model.

4.5 Prompt as specification

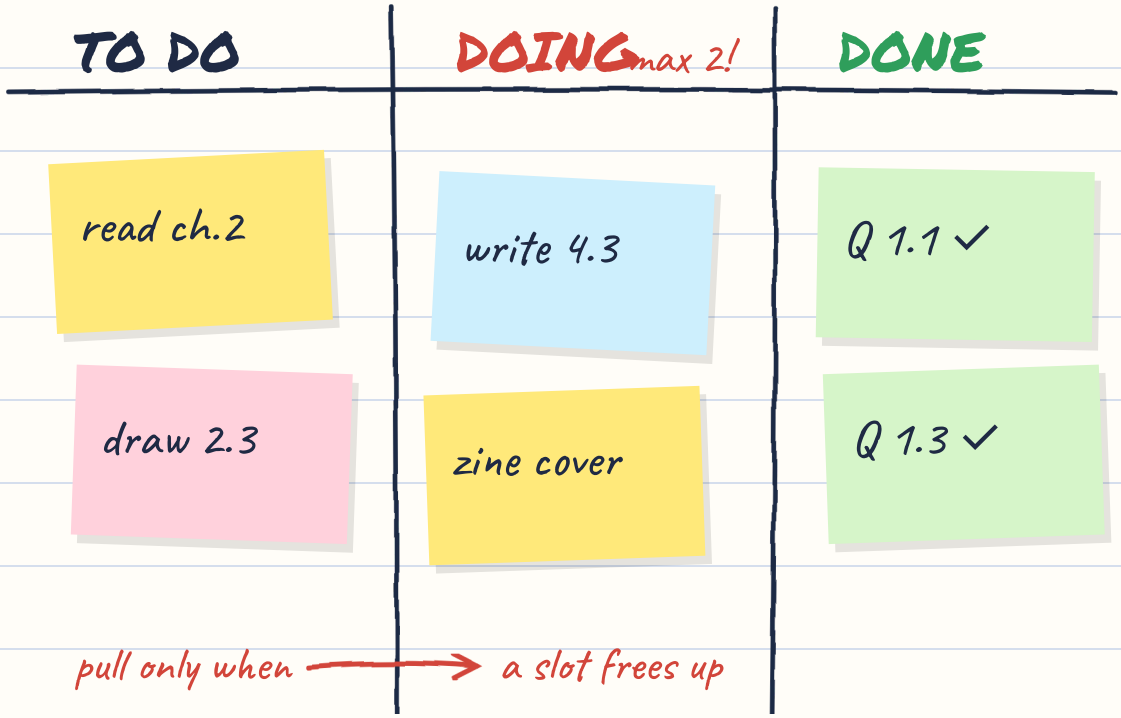
Why did we treat a prompt as a kind of writing rather than a trick? What makes a wrong answer recognizable?



We treated a prompt as writing because a prompt is a specification of intent, not a magic trick — in Karpathy's Software 3.0 the prompt is the program, so results depend on clear goals, context, audience, constraints and format, and tricks don't transfer between models. A wrong answer becomes recognizable when you define Success criteria before generating (Anthropic's first step): then each output can be checked against them and you can say exactly where it failed; without them a fluent wrong answer looks as convincing as a right one.

5.1 Kanban

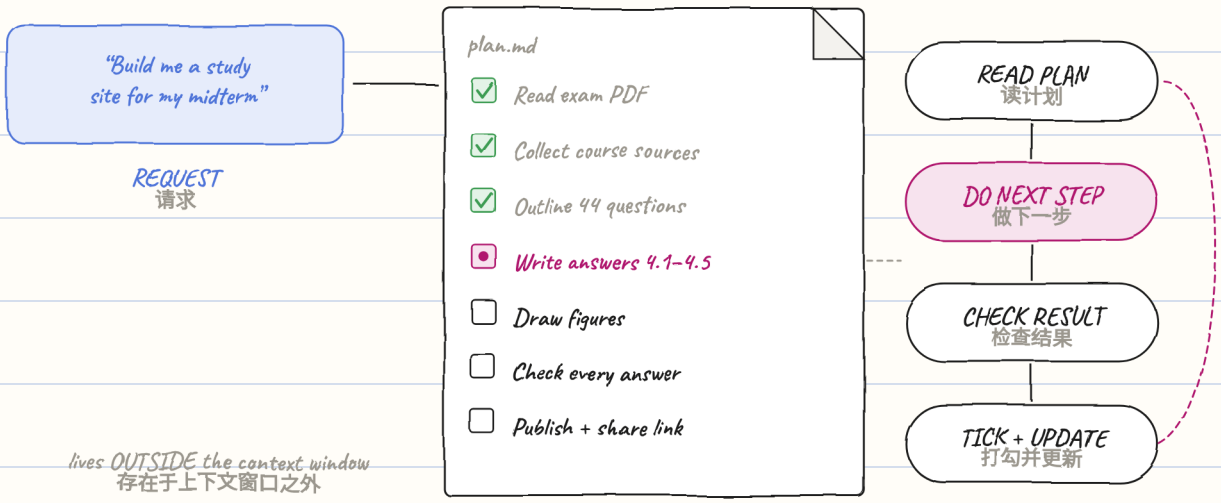
What is a Kanban board, and what can it track?



A Kanban board makes a workflow visible: columns are the stages work moves through (Backlog → Doing → Review → Done) and each card is one piece of work. Its defining rule (Ohno's signal card; Anderson; Kniberg & Skarin) is a WIP limit per column, so new work is pulled in only when a slot frees. It tracks the status of every task, who owns it, how much is in progress, where work piles up (bottlenecks), and lead time.

5.2 How an agent holds a request

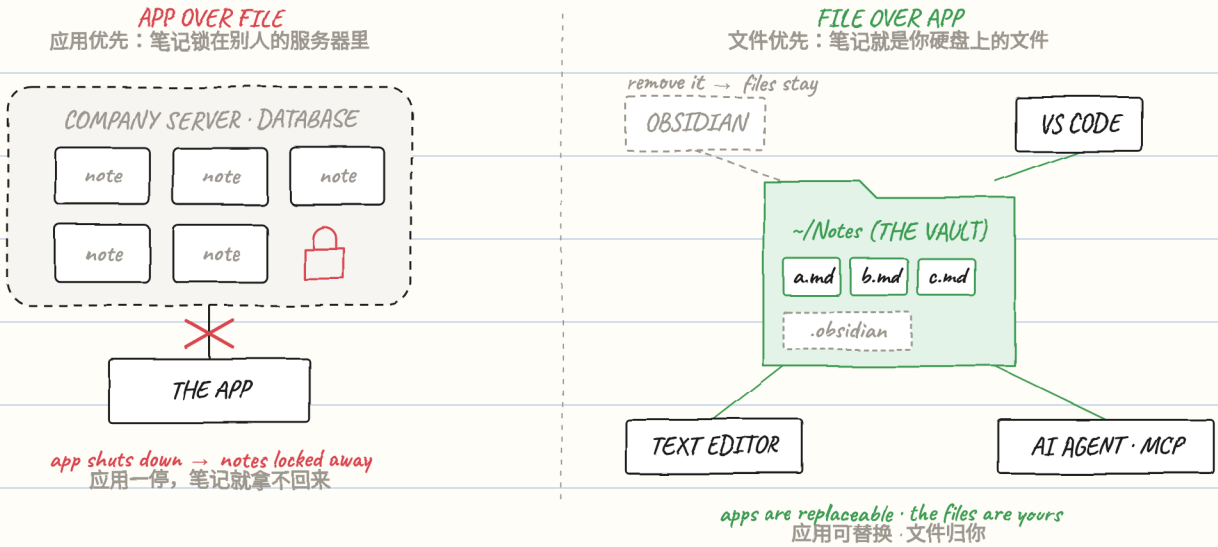
How can an agent keep track of a request (a checklist, a board, a plan)? Pick one and describe how it helps.



I pick the **plan / checklist**. The agent breaks the request into ordered steps and **writes them down** (a todo list or plan.md), then loops: read the plan → do the next step → check → tick it off and update. It helps because the plan is **memory outside the context window** (nothing is forgotten on a long task), it decomposes a vague request into verifiable steps, the agent can't claim "done" with boxes still empty, and the human can see and correct the plan before the work happens.

5.3 Vault / file over app

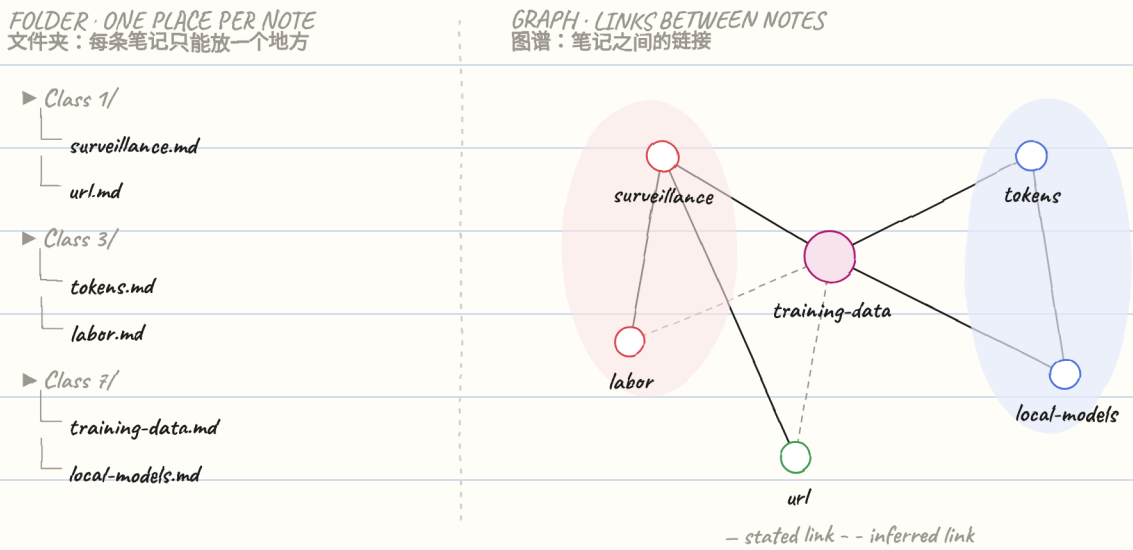
What is an Obsidian vault, and what does "file over app" mean for who owns your notes?



An Obsidian vault is just a folder on your own computer of plain-text Markdown (.md) files (plus a hidden .obsidian settings folder) — no proprietary database. Steph Ango's "file over app": apps are ephemeral, so lasting work must be files you control, in formats that are easy to retrieve and read. So you own your notes, not the app or the company: you can switch apps, sync any way, or let an agent read them, and they stay readable for decades.

5.4 Knowledge graph

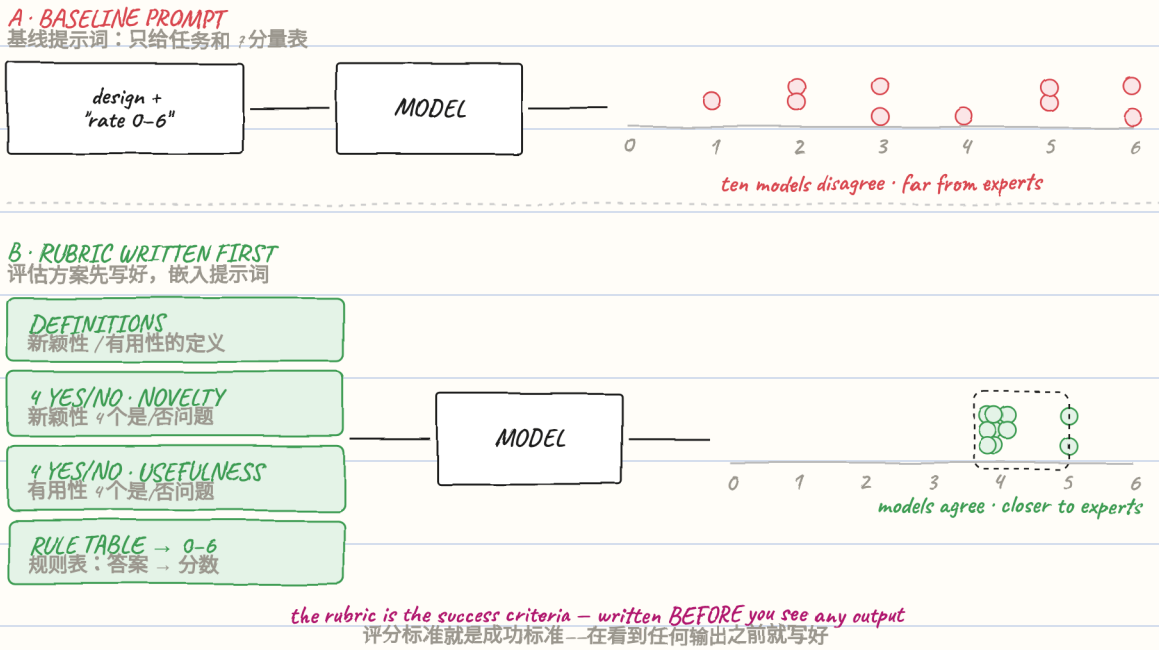
What does a graph of your notes show that a folder does not?



A folder is a hierarchy — each note sits in one place, so it shows only where a note is filed. A graph is a network of notes (nodes) and links (edges), so it shows how ideas relate: connections across folders, hubs that many notes point to, clusters of related notes (Graphify finds "communities" with the Leiden algorithm), orphan notes and gaps. Tools like Graphify make it queryable and mark which links were stated vs. inferred.

5.5 Rubric before generation

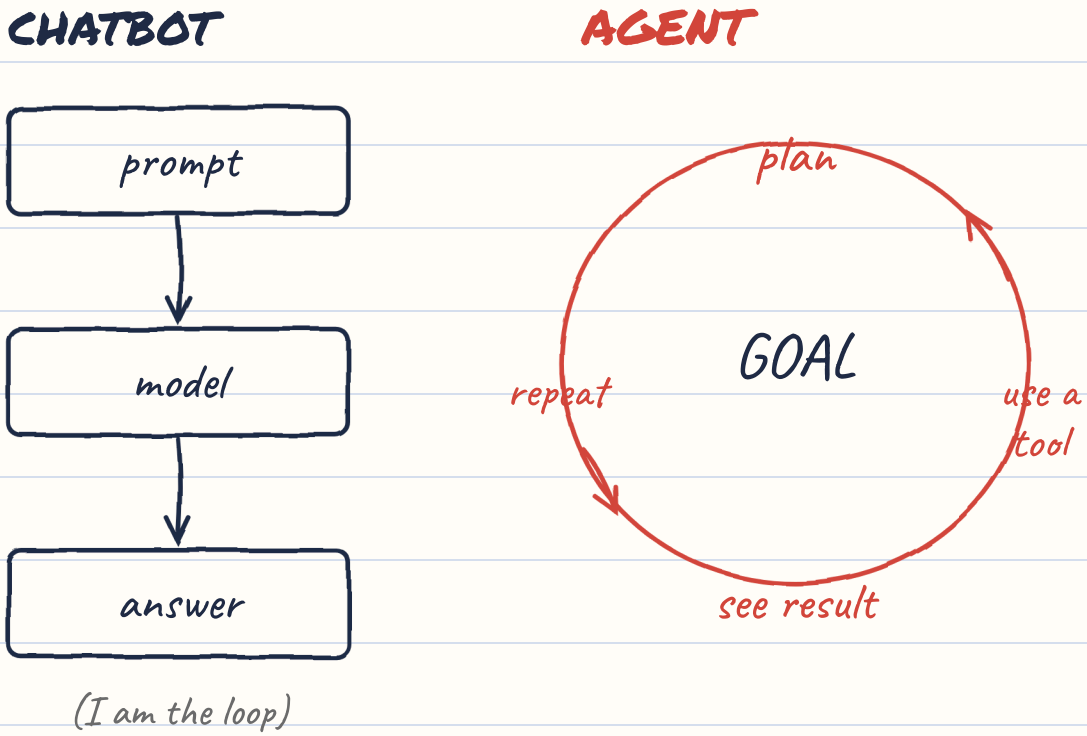
Why write the rubric before asking a model to evaluate creative work?



Because "creative" doesn't define itself — without criteria the model scores by its own hidden standard. Zhang, Han & Ahmed-Kristensen compared a baseline prompt (task + 0–6 scale) with a rubric written into the prompt: definitions of novelty and usefulness, four YES/NO questions for each, and a rule table mapping answers to a score. Across ten LLMs the rubric made scores more consistent and generally closer to expert ratings. Writing it first keeps the standard yours, not shaped by whatever output comes back.

6.1 Agent

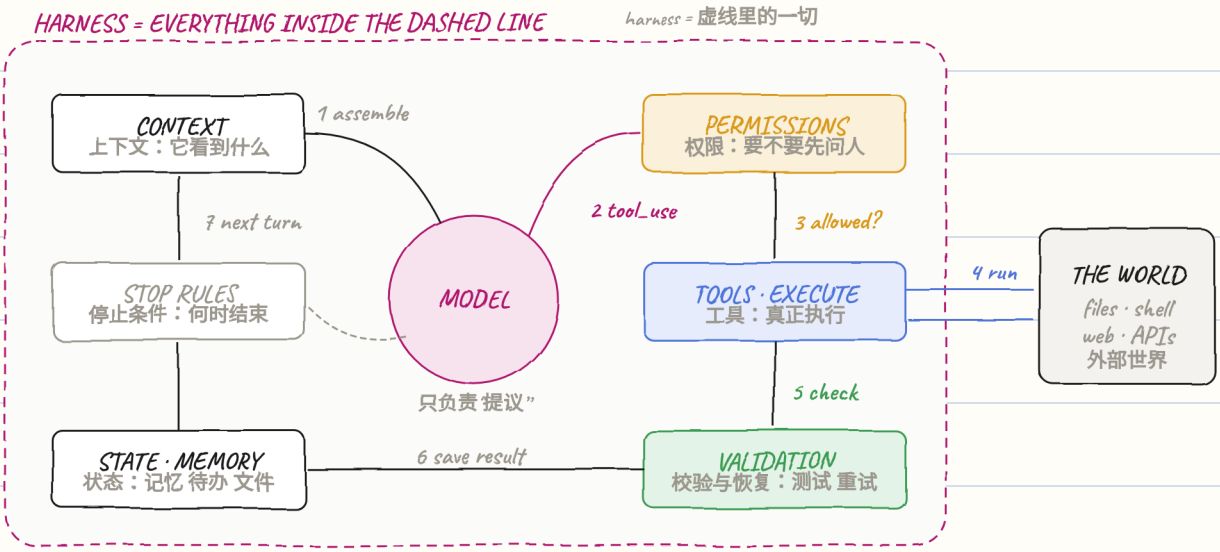
What makes a system an agent rather than a chatbot?



A chatbot answers one prompt at a time and cannot act — the human decides every next step. Anthropic defines an agent as a system where the LLM dynamically directs its own process and tool use: in practice an LLM using tools in a loop, based on feedback from its environment. So the difference is tools (it can act), the loop (it reads real results and decides the next step) and autonomy over the path (you give a goal; it plans and decides when it's done). Fixed, coded steps are a workflow, not an agent.

6.2 Harness

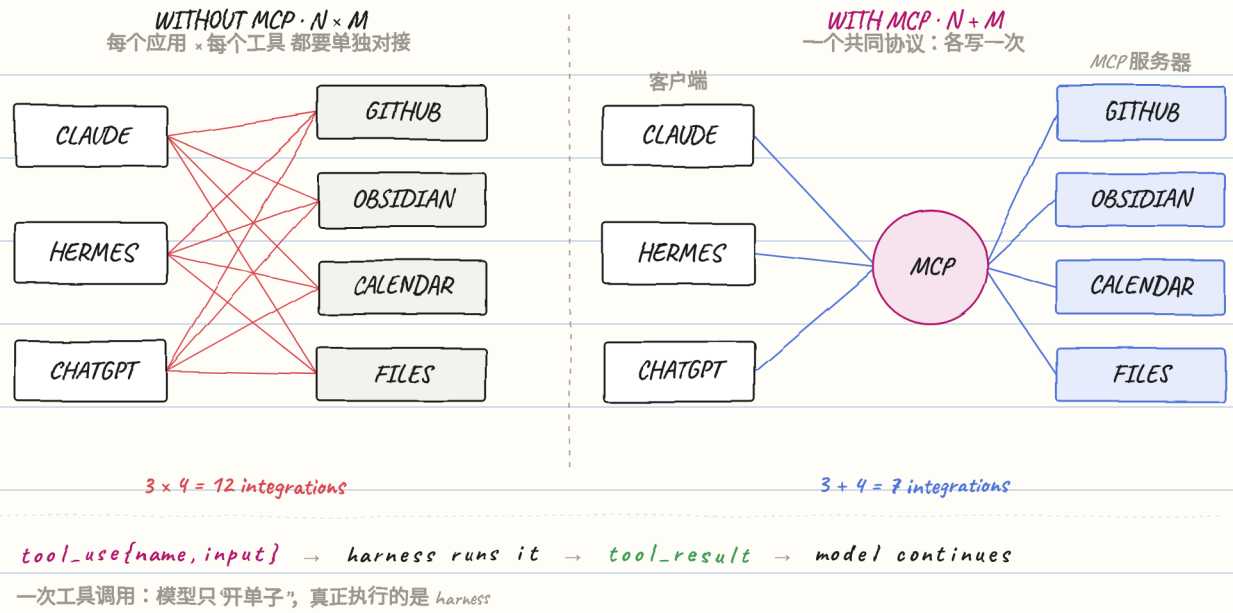
What is a harness (Hermes, Claude Code), and how does it use a model?



A harness is the software around a model that turns it into an agent. Pan et al. define it as the external execution system that decides what the model sees, which tools it may call, where state is stored, when validation runs, how failures are recovered, and when execution stops. Hermes and Claude Code are harnesses, not models. It uses the model in a loop: build the context → the model proposes text or a tool call → the harness checks permission, runs the tool, feeds the result back → repeat until a stop rule fires.

6.3 Tool use / MCP

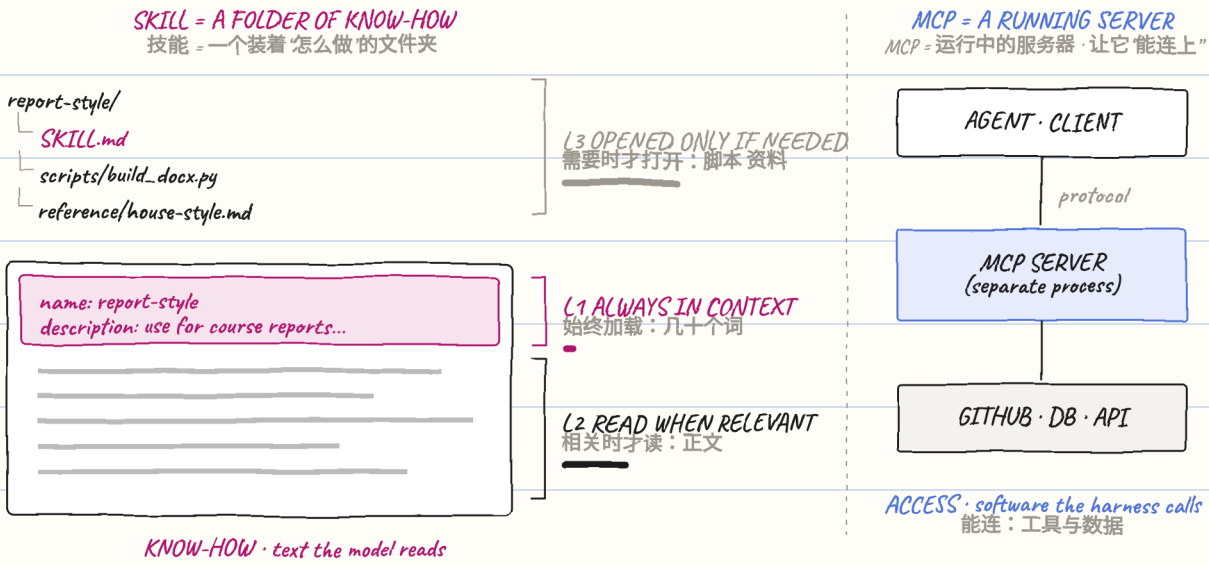
What is a tool call, and what problem does the Model Context Protocol solve?



A **tool call** is a structured request the model writes — a tool name plus JSON arguments (a tool_use block). The model never runs it; the **harness executes it and returns a tool_result**, which the model reads before its next step. **MCP** (Model Context Protocol) is an **open standard** for connecting AI apps (clients) to tools and data (servers). It solves the **N × M integration problem**: instead of every assistant building a custom connector for every service, each implements the protocol once — N + M.

6.4 Skill

What is a 'skill', and how is it different from an 'MCP' server?



A skill is a folder of know-how: a SKILL.md file (name, description, step-by-step instructions) plus optional scripts and reference files. It loads by progressive disclosure — only the name and description sit in context; the instructions are read when the task matches, extra files only when needed. An MCP server is a separately running program that gives the agent access to new tools and data over a protocol. Skill = how to do it (text the model reads); MCP = what it can reach (software the harness calls).

6.5 Permission model / delegation

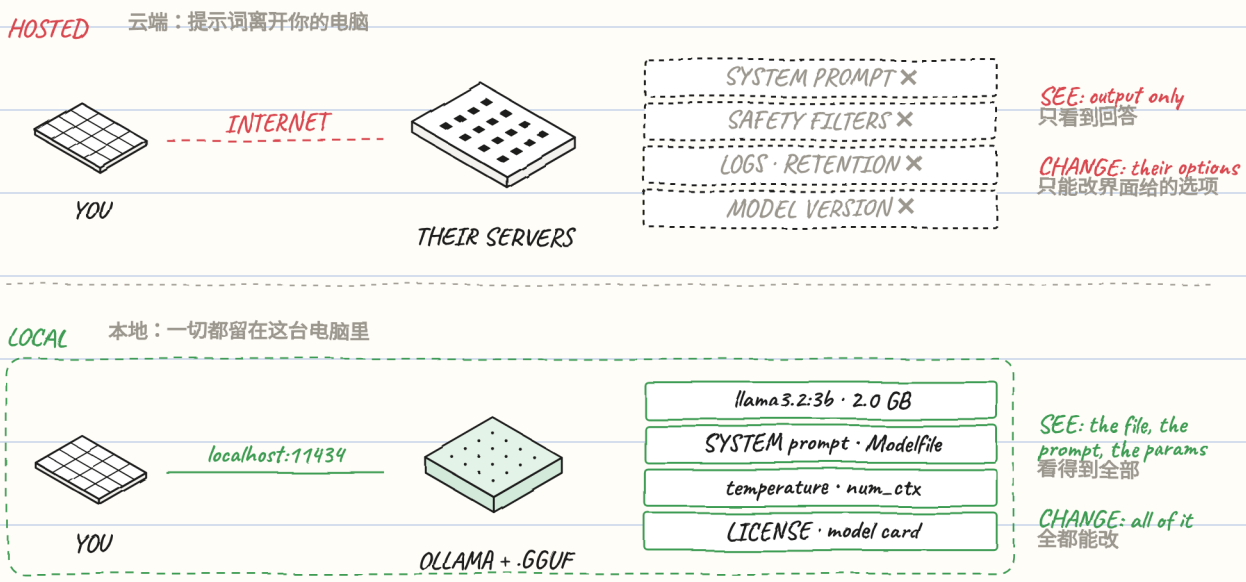
What runs without asking, what stops for approval, what is logged afterward: why is this the central design decision in an agentic product?



Every action falls into three tiers: runs without asking (safe, reversible — read, search), stops for approval (irreversible, external or costly — edit/delete files, shell commands, push, send, spend) and logged afterward (transcript, tool calls, diffs, cost) so it can be reviewed and undone. It is the central design decision because the agent acts with your authority: the line sets the trade-off between usefulness and safety (too many prompts → approval fatigue; too few → one mistake or prompt injection does real damage), who is accountable, and how autonomy grows with trust — Karpathy's autonomy slider.

7.1 Local vs. hosted

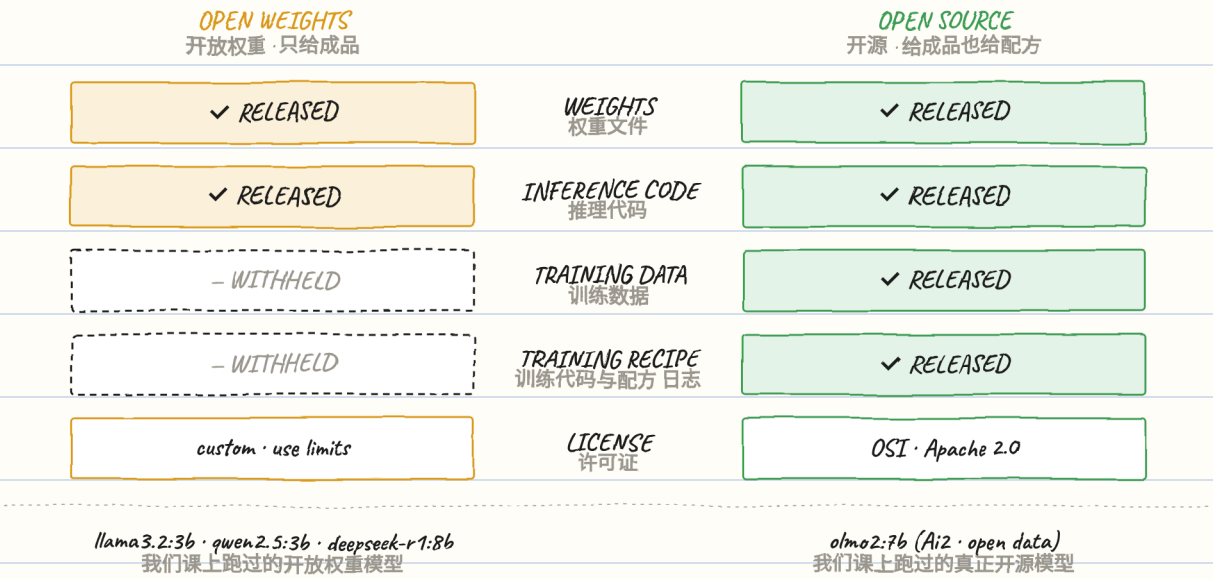
When the same prompt goes to a hosted model and to a model on your laptop, what changes about where it goes, what you can see, and what you can change?



With a hosted model the prompt leaves my laptop and crosses the Internet to the company's servers, where a hidden system prompt, filters and logging act on it; I see only the output and can change only the settings the interface offers. With a local model (Ollama) the prompt goes to `localhost` and never leaves the machine; I can see the weights file, Modelfile, parameters and license and change all of it (swap models, rewrite the system prompt, temperature, quantization). Trade-off: hosted is more capable; local gives privacy and control but is limited by my hardware.

7.2 Open weights vs. open source

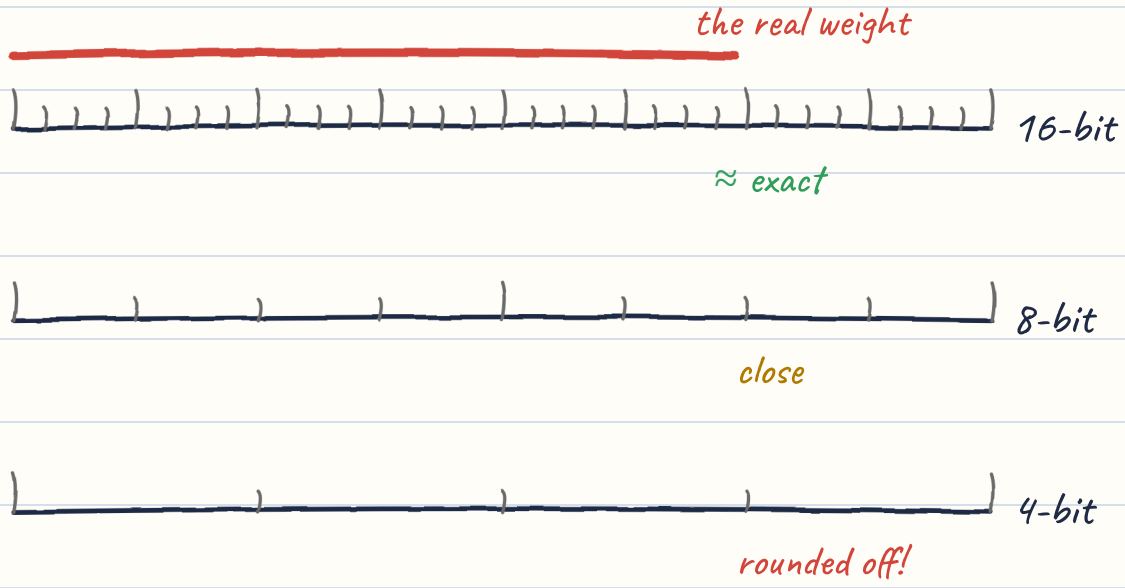
What is the difference? Name one model you ran in each category and its license.



Open weights means the trained weights are downloadable to run and fine-tune, but the training data and recipe are withheld, often under a custom license with use limits. Open source means weights, training data, code and recipe are all released under an OSI-approved license, so you can study and rebuild the model. Open weights: `llama3.2:3b` (Meta, Llama 3.2 Community License). Open source: `olmo2:7b` (Ai2, Apache 2.0, with open training data, code and checkpoints).

7.3 Quantization

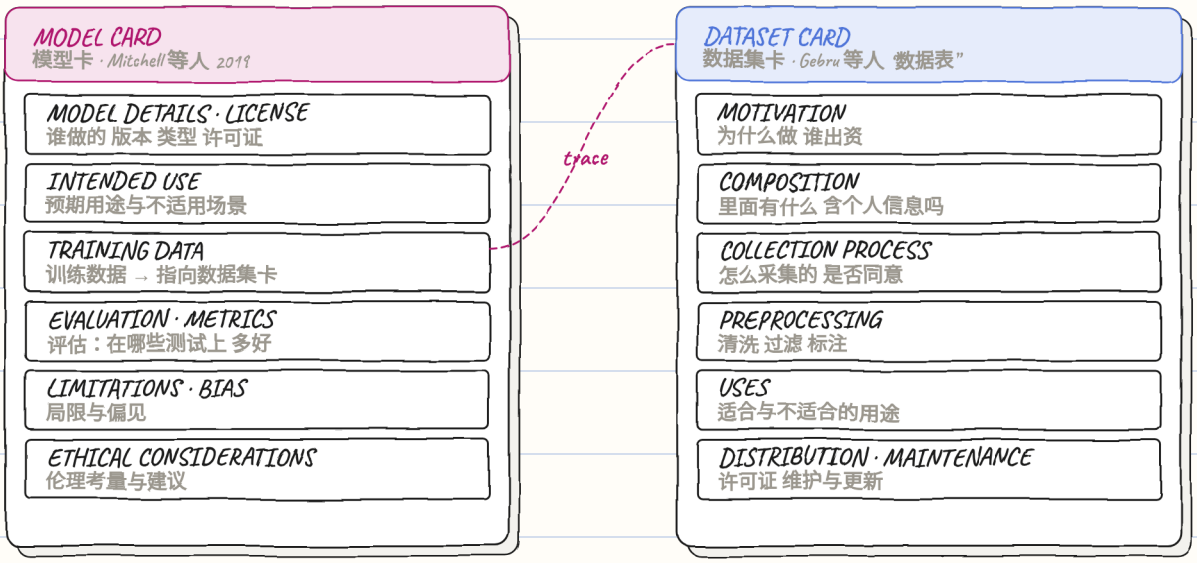
What is quantization, and what do you gain and lose by running a smaller version of a model?



Quantization stores each weight with fewer bits (16-bit → 8 or about 4) by rounding it onto a coarser scale. Gain: a much smaller file (Llama 3.2 3B: 6.4 GB → 2.0 GB at Q4_K_M) that fits in a laptop's memory and runs faster, so the model can run locally. Lose: precision — a small error in every weight, which shows up as more mistakes in reasoning, math and long or detailed tasks (Q8 ≈ lossless, Q4 a good compromise, Q2 clearly worse).

7.4 Model card / dataset card

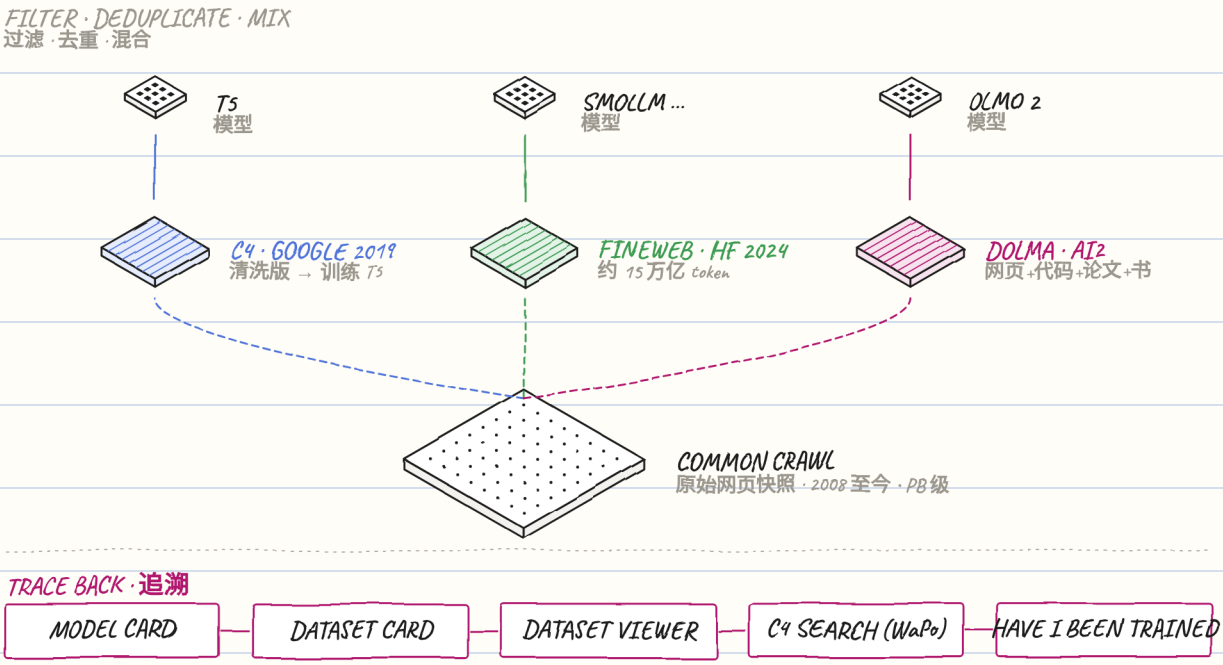
What is each one supposed to disclose?



A model card (Mitchell et al., 2019) discloses the model: who made it and its license, intended and out-of-scope uses, its training data, evaluation results broken down across groups, and limitations, biases and ethical considerations. A dataset card / datasheet (Gebru et al.) discloses the data: motivation (why, by whom, funded by whom), composition (what's in it, any personal data), collection process and consent, preprocessing, recommended uses, distribution and maintenance.

7.5 Training corpus

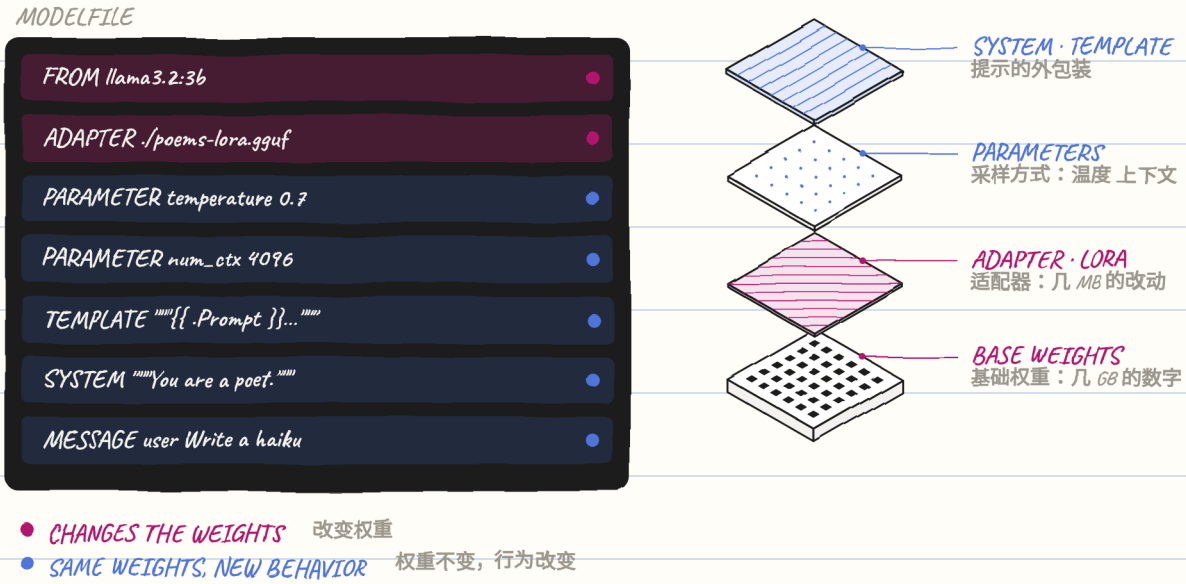
Common Crawl, C4, FineWeb, Dolma: how do these relate, and how do you trace a model back to its data?



Common Crawl is the raw web scrape (monthly snapshots since 2008). C4, FineWeb and Dolma are filtered, deduplicated cuts of it: C4 (Google, built to train T5), FineWeb (Hugging Face, ~15 trillion tokens from 96 snapshots), Dolma (Ai2, Common Crawl mixed with code, papers, books and Wikipedia, for OLMo). To trace a model back: model card → its training-data section → the dataset card → the dataset viewer → search the corpus itself (Washington Post C4 search, Have I Been Trained).

8.1 Modelfile

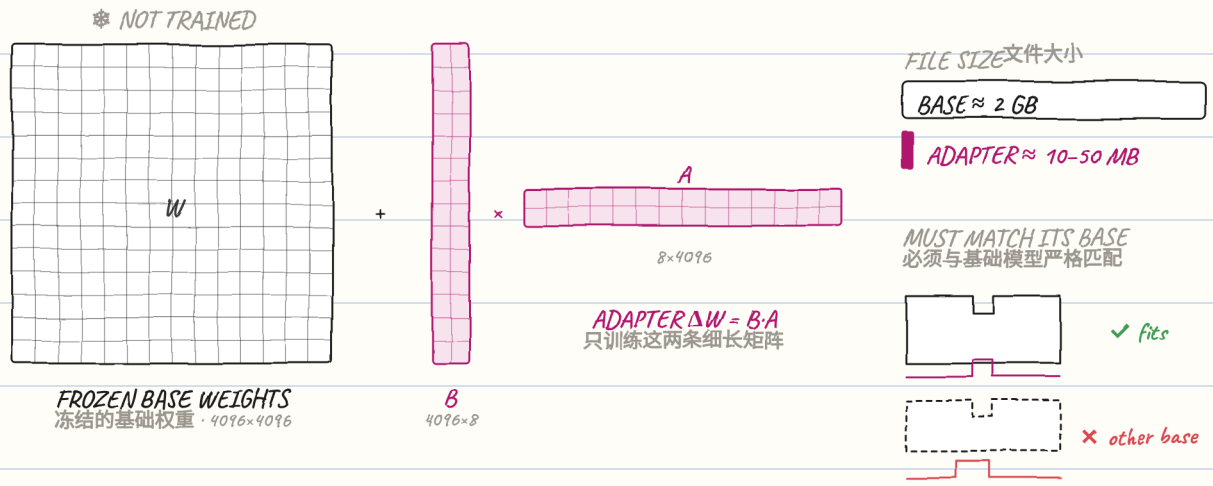
What is a Modelfile, and which of its lines change the weights and which do not?



A Modelfile is Ollama's plain-text blueprint for building a model (`ollama create name -f Modelfile`). Only FROM (which base weights you start from) and ADAPTER (a fine-tuned LoRA applied on top) change the weights. PARAMETER (temperature, num_ctx...), TEMPLATE, SYSTEM, MESSAGE and LICENSE only change how the same weights are prompted, sampled or labeled.

8.2 Fine-tuning / LoRA adapter

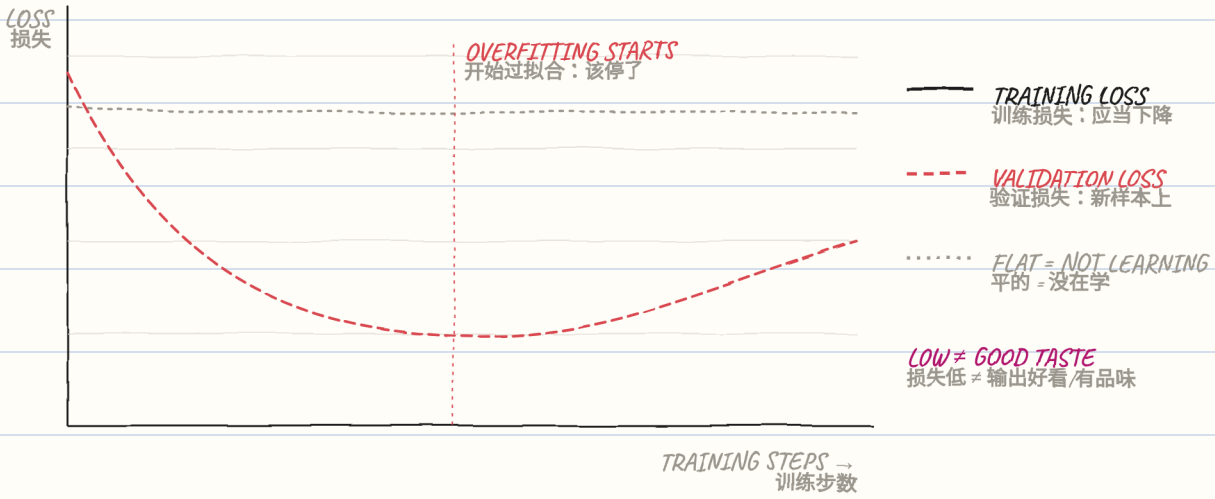
What is an adapter, why is it small, why must it match its base, and what does it change well and badly?



An adapter (LoRA) is a small set of learned changes added on top of a frozen base model: for each big weight matrix it trains two thin, low-rank matrices whose product ΔW is added in. It is small because the rank is tiny (≈ 8 vs. 4,096) — megabytes, not gigabytes. It must match its base because it stores differences relative to those exact weights and shapes (otherwise it fails or behaves erratically). It changes style, tone and format well, and new facts or reasoning badly (and overfits small data).

8.3 Loss

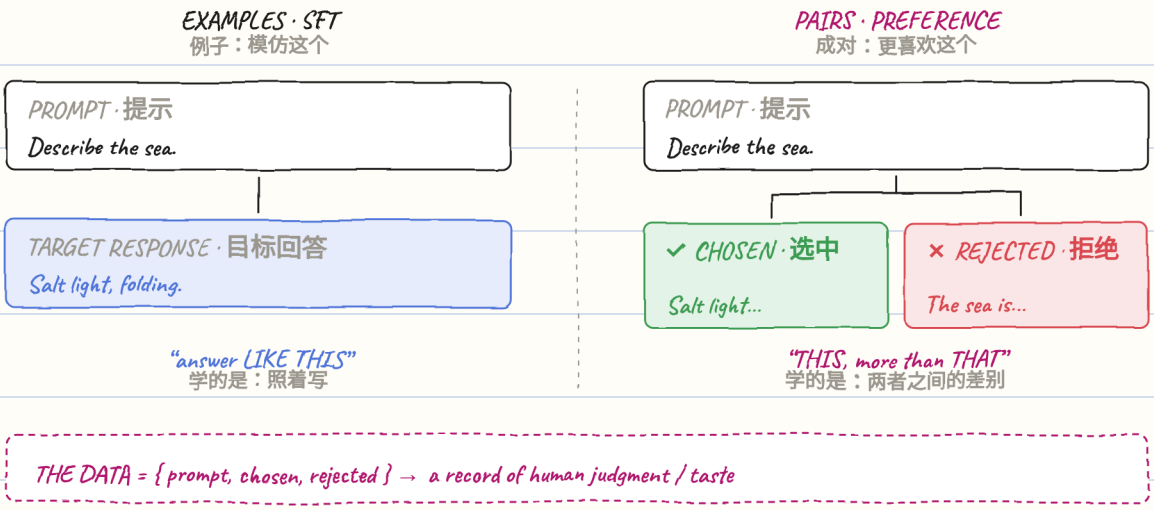
What does the loss number tell you during training?



Loss is a running measure of how wrong the model's next-token predictions are on the training examples (cross-entropy: how surprised it is by the right token). Falling loss means it is learning to imitate the data; flat means it isn't learning; falling toward zero on a small dataset while validation loss rises means it is memorizing (overfitting). It does not measure quality or taste — you still have to read the outputs.

8.4 Preference tuning

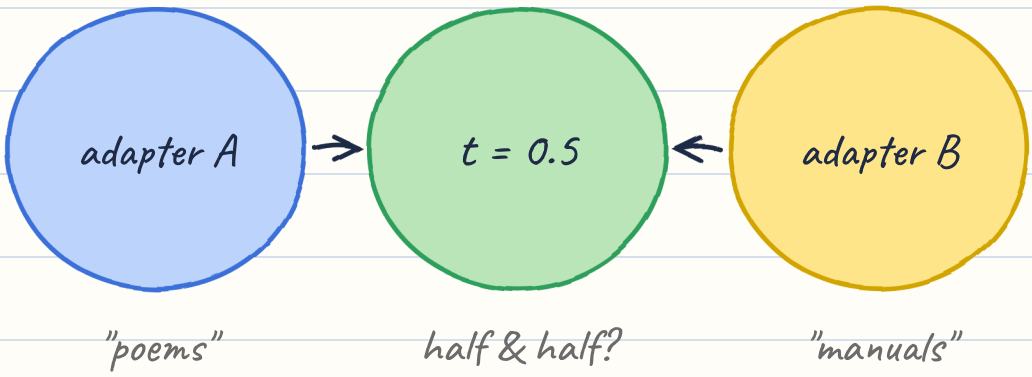
How is training on chosen/rejected pairs different from training on examples? What is the data, in that case?



Training on examples (SFT) shows the model one target answer to imitate: it learns "answer like this." Training on chosen/rejected pairs (DPO, or RLHF's reward model) shows two answers to the same prompt and pushes the chosen one up relative to the rejected one: it learns what to prefer, not what to copy. The data is rows of {prompt, chosen, rejected} — a record of human judgment and taste.

8.5 Merge / blend

What happens when two adapters are blended at t = 0.5, and who wrote the result?



$$W = (1-T)\cdot A + T\cdot B$$

Blending is arithmetic on the weights: $W = (1 - t)\cdot A + t\cdot B$, so at $t = 0.5$ every number is the exact midpoint of the two adapters (trained on the same base). The model behaves somewhere between the two learned behaviors — a hybrid, one style dominating, or mush — because the network is not linear, so you must read the outputs. No one wrote the result directly: authorship is distributed across the people behind datasets A and B, the base model's makers and its data, and whoever chose the adapters and t.

9.1 When the guidelines apply

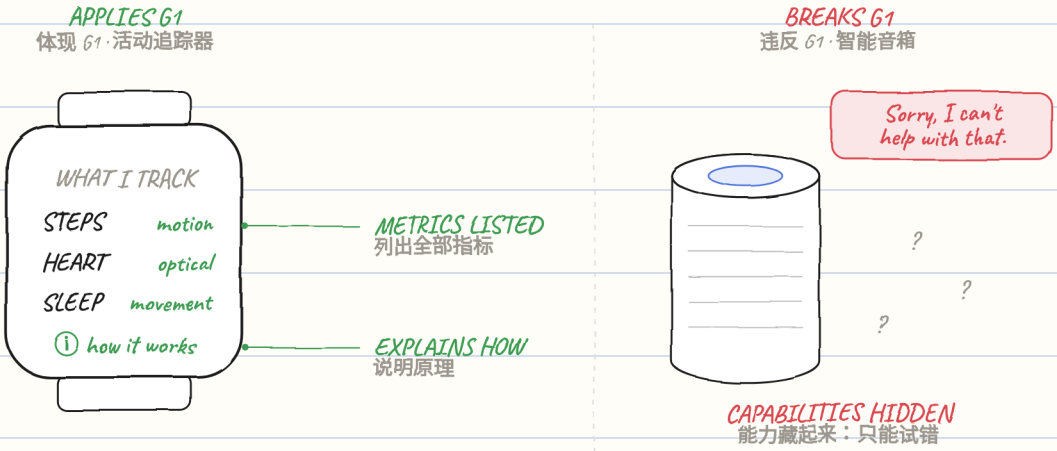
The 18 guidelines are grouped by when they apply: Initially, During interaction, When wrong, and Over time. Choose one guideline from each group and say in your own words what it asks of the designer.



Amershi et al. group the 18 guidelines by when they apply. Initially — G1 Make clear what the system can do: show users the AI's scope before they rely on it. During interaction — G4 Show contextually relevant information: surface only what fits the user's current task and setting. When wrong — G9 Support efficient correction: assume errors and make editing or undoing cheap. Over time — G17 Provide global controls: let users set what the system monitors and how it behaves overall.

9.2 An example application

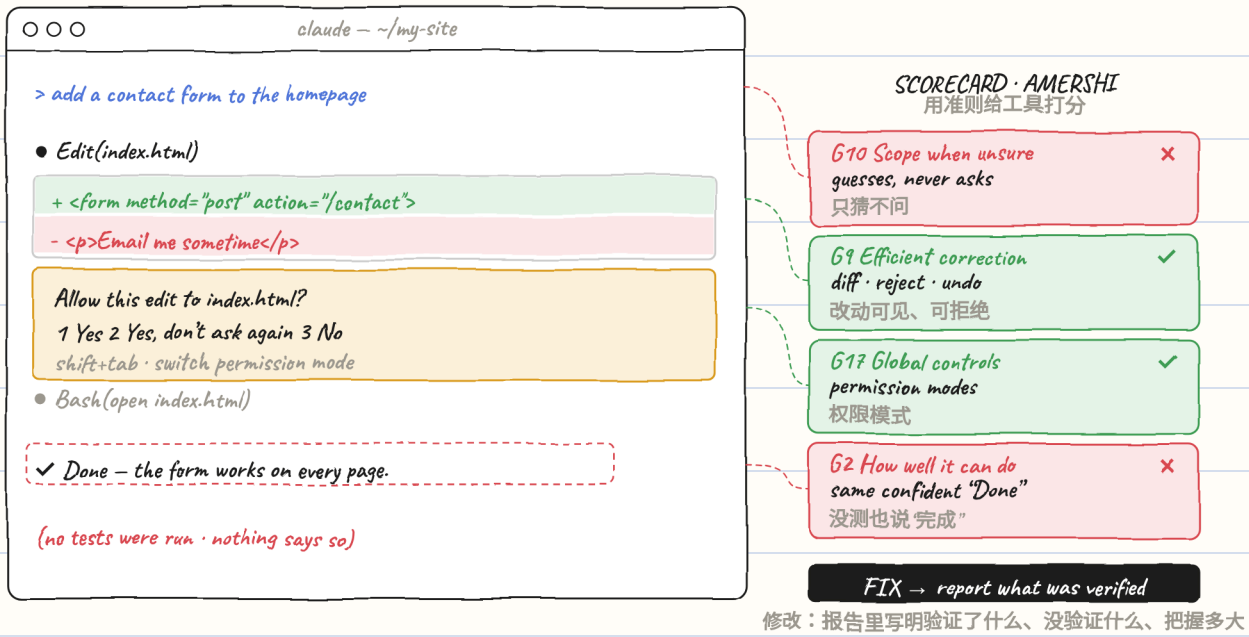
Each guideline comes with an example from a product category the participants tested (activity trackers, music recommenders, navigation, voice assistants, autocomplete, email and others). Pick one example and explain why it applies its guideline. Then describe a product you use that breaks the same guideline.



In Table 1, G1 – Make clear what the system can do is illustrated by activity trackers: the product displays all the metrics it tracks and explains how. That applies G1 because users learn the system's scope before trusting it, so their expectations match what it can really do. A smart speaker (e.g. Alexa) breaks G1: it shows no list of abilities, so I only discover its limits when it answers "Sorry, I can't help with that."

9.3 The guidelines and our tools

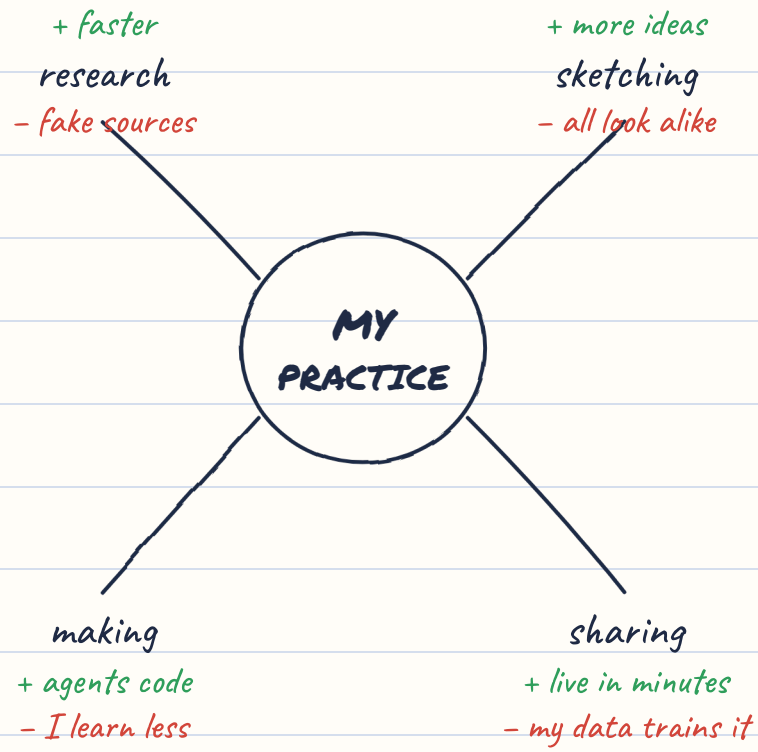
Pick one tool we used this term (a commercial assistant, an agent harness, a local model, Variable Machine). Which guideline does it apply well, which does it break, and what would you change?



Claude Code (an agent harness) applies G17 – Provide global controls: its permission modes let me decide once what it may do without asking, and every edit appears as a diff I can reject (G9, efficient correction). It breaks G2 – Make clear how well the system can do what it can do: it reports "Done" with the same confidence whether or not it ran any tests. I would add a verification summary to every report — what ran, what was assumed, what is untested — with a confidence level, and have it ask before guessing.

C5 What would you do with it

Where in your practice do these technologies and concepts resonate the most? Describe this in both positive and negative aspects.



These tools resonate most in the making stage of my design work. Positive: a local model and an agent harness let me prototype in an afternoon — I write a clear spec (prompt as specification), the agent builds, and I check it against a rubric I wrote first; with file over app my notes and sketches stay mine. Negative: the same speed risks de-skilling and sameness — if every idea starts from the same model, the work starts to look alike; and hosted tools feed surveillance capitalism with my drafts. My rule: research and ideas by hand first, AI for building, and local models for anything private.

BEFORE I CLIP IT:

- [x] every question answered (no blanks!)
- [x] definitions as the SOURCES say them
- [x] source named (Wright, Zuboff, MDN…)
- [x] in MY own words, not copied
- [x] one drawing / collage per class
- [x] name + section on the cover
- [x] pages in order, binder clip on

due Fri Nov 20 · 5 pm